CTF Write-Up - 2

February 3, 2026

CS 431

Computer and Network Security
Prof. Abhishek Bichhawat

Indian Institute of Technology Gandhinagar
Palaj, Gandhinagar - 382355

CTF 2 (UName: 22110089)

Submitted by

Guntas Singh Saran (22110089)
Computer Science and Engineering

Problem 5: rsa-state-of-affairs

Flag

`CNS431{r5a_5t4t3_0f_4ff41r5}`

Vulnerability:

Given a very small public exponent ($e = 5$) and no padding, when RSA is used the ciphertext becomes simply m^e without modular reduction.

How it can be exploited & steps:

Since the encrypted cipher is $c = m^e \bmod n$, (c is the remainder when m^e is divided by n), so if $m^e < n$ (which is the case with small exponent e), the remainder is m^e itself, so $c = m^e$. The original message hence can be recovered by computing the integer e-th root of the ciphertext.

Code

```
c = 83978071....523448832

# take the 5th root of c using integer arithmetic
def nth_root(n, root):
 high = 1
 while high ** root < n:
  high *= 2
 low = high // 2
 while low < high:
  mid = (low + high) // 2
  mid_root = mid ** root
  if mid_root < n:
   low = mid + 1
  elif mid_root > n:
   high = mid
  else:
   return mid
 return low

m = nth_root(c, 5)
print(m.to_bytes((m.bit_length() + 7) // 8, 'big').decode())
```

Problem 6: Alice's Signature Mix-Up

Flag

`CNS431{@l1c3_m@d3_@_k3y_mist@k3}`

Vulnerability:

Alice used an extremely weak *additive hash* instead of a cryptographic hash. She then attempted to "sign" this hash using RSA but **mistakenly used the public key instead of the private key**.
An additive hash is computed by simply summing the byte values of the message, which provides

no security. The challenge was that I had spent a lot of time using standard hashing methods like SHA-1, SHA-256, and MD5, but the hint highlighted the naivety of the hashing method.

How it can be exploited:

- Convert the message to bytes.
- Compute the additive hash by summing all byte values.
- Compute the signature as $s = h^e \bmod n$ using the public key.

Code

```
n = 1013.....639
e = 65537
secret_message = b"Hello Bob! This is a secret message, to be kept within CNS431."

h = sum(secret_message)
print(f"Hash of secret message: {h}")

# s = h^d mod n (where d is Alice's private key)
# but she mixed up and used e instead of d
# so s = h^e mod n
s = pow(h, e, n)
print(f"s: {s}")
```

Problem 7: hash-extension

Flag

`431fl@g{D3@D3Xt3ns!0nL3ngth}`

Vulnerability:

The server authenticated cookies using the SHA-256 as Message Authentication Code (MAC). This is insecure because SHA-256 is vulnerable to a **length extension attack** when used as a MAC.

Why this works:

SHA-256 processes data block-by-block (512 bits) and exposes its internal state as the final hash. If an attacker knows the hash of a message and the length of the secret key, they can continue hashing additional data without knowing the key. This allows forging a valid hash for an extended message. Which means if `SHA256(K || M)` is known, we can compute `SHA256(K || M || padding || extra)`.

How I exploited it:

- Original cookie:

  `username=cns431&groups=students,users,`

- What **we want** is to add ourselves to the `admins` group, so tampered cookie:

  `username=cns431&groups=students,users,admins,`

- we are told that `SECRET_KEY` is of length 16, and the **signature** of the above original cookie (message) is computed as:

```
signature = SHA256(SECRET_KEY || message)
```

= `c99963ab62fa272f73238a3e79f0f21d9837fa33cc34c8ae094a72d5110823ef`

- so this hash/signature is literally the internal SHA-256 state after hashing:

```
SECRET_KEY || message || padding
```

- what needs to be done is figure out the correct padding for SHA-256 and append `admins,` - to do this, I used a tool hashpump: github.com/2H-K/hashpumpy_changed
- this tool treats our given signature as a SHA-256 internal state, takes in the original message, the key-length and recreates the padding, appends to our message and produces the new valid signature and the new message.

Code

```
$ git clone --depth=1 https://github.com/2H-K/hashpumpy_changed
$ apt-get install g++ libssl-dev
$ cd hashpumpy_changed/
$ make
$ make install
$ hashpump --help
> HashPump [-h help] [-t test] [-s signature] [-d data] [-a additional] [-k keylength]
    HashPump generates strings to exploit signatures vulnerable
    to the Hash Length Extension Attack.
    -h --help          Display this message.
    -t --test          Run tests to verify each algorithm is operating properly.
    -s --signature     The signature from known message.
    -d --data          The data from the known message.
    -a --additional    The information you would like to add to the known message.
    -k --keylength     The length in bytes of the key being used to
    sign the original message with.
    Version 1.2.0 with CRC32, MD5, SHA1, SHA256 and SHA512 support.
    <Developed by bwall(@botnet_hunter)>


$ hashpump \
  -s c99963ab62fa272f73238a3e79f0f21d9837fa33cc34c8ae094a72d5110823ef \
  -d "username=cns431&groups=students,users," \
  -a "admins," \
  -k 16
> b977414d7bc068b298b08f842a78b51399254c73e704427f2078b59491102c65
  username=cns431&groups=students,users,\x80\x00\x00\x00\x00\x00\x00\x00\x01\xb0admins,
```

- So we get our new cookie and new signature, encoded the cookie to hex using python:

```
cookie= b"username=cns431&groups=students,users,\x80\x00\x00\x00\x00
\x00\x00\x00\x01\xb0admins,"
print(cookie.hex())
# 757365726e616d653d636e733433312667726f7570733d73747564656e74732c75736
572732c8000000000000000001b061646d696e732c
```

```
❯ nc 10.0.118.104 4310

Welcome to the CNS431 Hash Extension Attack Challenge!
Hold on to your hats, because this is going to be a wild ride!
We now use SHA256 as a message-authentication code to sign and authenticate users' cookies!

Here is your cookie as plaintext:
username=cns431&groups=students,users,
And here it is as hex:
757365726e616d653d636e733433312667726f7570733d73747564656e74732c75736572732c

SECRET_KEY is a 16-byte key that is unknown to you
The signature is computed as SHA256(SECRET_KEY || message)
The signature of your cookie is:
c99963ab62fa272f73238a3e79f0f21d9837fa33cc34c8ae094a72d5110823ef

Please provide your cookie encoded as hex:
757365726e616d653d636e733433312667726f7570733d73747564656e74732c75736572732c8000000000000000001b061646d696e732c
Please provide the signature SHA256(SECRET_KEY || message) of the cookie:
b977414d7bc068b298b08f842a78b51399254c73e704427f2078b59491102c65
Decoded cookie: b'username=cns431&groups=students,users,\x80\x00\x00\x00\x00\x00\x00\x00\x01\xb0admins,'
Signature verified
Welcome admin! Here is your flag:
431fl@g{D3@D3Xt3ns!0nL3ngth}
Goodbye
```

Figure 1: Submitting to get the flag.

Problem 8: Login Error

Flag

`cnsctf{22110089_1770070630.6744974_4ded1d3b8ab00a92f7e72766b5fda132b8ca95f655efa328ca23a4a5167a9283}`

Vulnerability:

AES-CBC bit-flipping attack: The program uses AES encryption in CBC mode but does not authenticate the ciphertext. In CBC mode, the first plaintext block is obtained by XORing the decrypted ciphertext with the Initialization Vector (IV).

$$P_1 = D_K(C_1) \oplus IV$$

This means that if an attacker modifies the IV, the decrypted plaintext will change in a predictable way, even without knowing the secret encryption key.

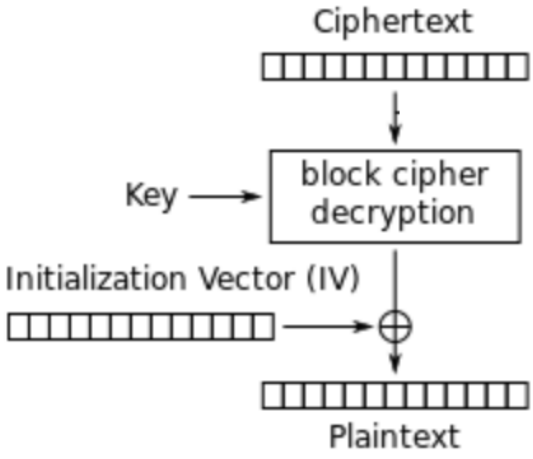

Figure 2: A single block of AES-CBC. Souce: easefilter.com/kb/understand-aes-encryption.htm

Steps to exploit:

- First, I observed that the error message mentioned CBC mode, which confirmed that AES-CBC was being used.

```
$ nc 10.0.118.104 9999
Welcome to our super cool AES-encryption portal!
Oops! We forgot the admin credentials, but we remember the username was
like c?s and the password was like c?f.
When we tried 'user:c?s' and 'pass:c?f', the portal gave us these hex strings:

e25dbc5241ad9fbe40d9aad97f9a57f79a584654fbe8e55a985be1317fe8bc66
e25dbc5241ad9fbe40d9aad97f9a57f786406e25dce45b81e516927db06d38a5

To unlock the portal, enter the correct hex strings that decrypt to the
right credentials. Good luck!
Enter your 8 digit roll no : 22110089
Enter username hex string : 636e73
Enter password hex string : 637466
Error 3 while decrypting in CBC mode
Error! Something went wrong. Try to connect again.
```

- Also, I noticed that every time the service was run, it returned two ciphertexts where the first 16 bytes (128 bits) were identical. This indicated that the **first block was the IV** and the **second block was the actual ciphertext**.

```
[ IV (128 bits) ][ C1 (128 bits) ]
```

- The server revealed that the decrypted credentials had the format `user:c?s` and `pass:c?f`, suggesting that a single unknown character was present in each string. Based on the context, I guessed that the correct credentials were `user:cns` and `pass:ctf`.

- As seen from how CBC works, if we want to change `P[i]` from `'?'` to `'n'`, we do:

$$\texttt{P[i] = D_K(C)[i]} \oplus \texttt{IV[i]}$$

$$\texttt{IV'[i] = IV[i]} \oplus (\texttt{'?'} \oplus \texttt{'n'})$$

- I calculated the XOR difference between the character `?` and the desired characters `n` and `t`, and applied this difference to the appropriate byte of the IV.
- Send this combined `modified IV || original ciphertext.`

Code

```
# d06b91365ae9f3e59308b84ba71b6e53 123b8dcac31578ea1c4ee5496fbbce0e
# d06b91365ae9f3e59308b84ba71b6e53 688109854f7dc49373de770bc6390dcc

iv = bytes.fromhex("d06b91365ae9f3e59308b84ba71b6e53")

# index of '?' in "user:c?s" and "pass:c?f"
IDX = 6

# Flip '?' → 'n'
uiv = bytearray(iv)
uiv[IDX] ^= 0x51

# Flip '?' → 't'
piv = bytearray(iv)
piv[IDX] ^= 0x4b

cu = bytes.fromhex("123b8dcac31578ea1c4ee5496fbbce0e")
cp = bytes.fromhex("688109854f7dc49373de770bc6390dcc")

print((bytes(uiv) + cu).hex())
print((bytes(piv) + cp).hex())
```

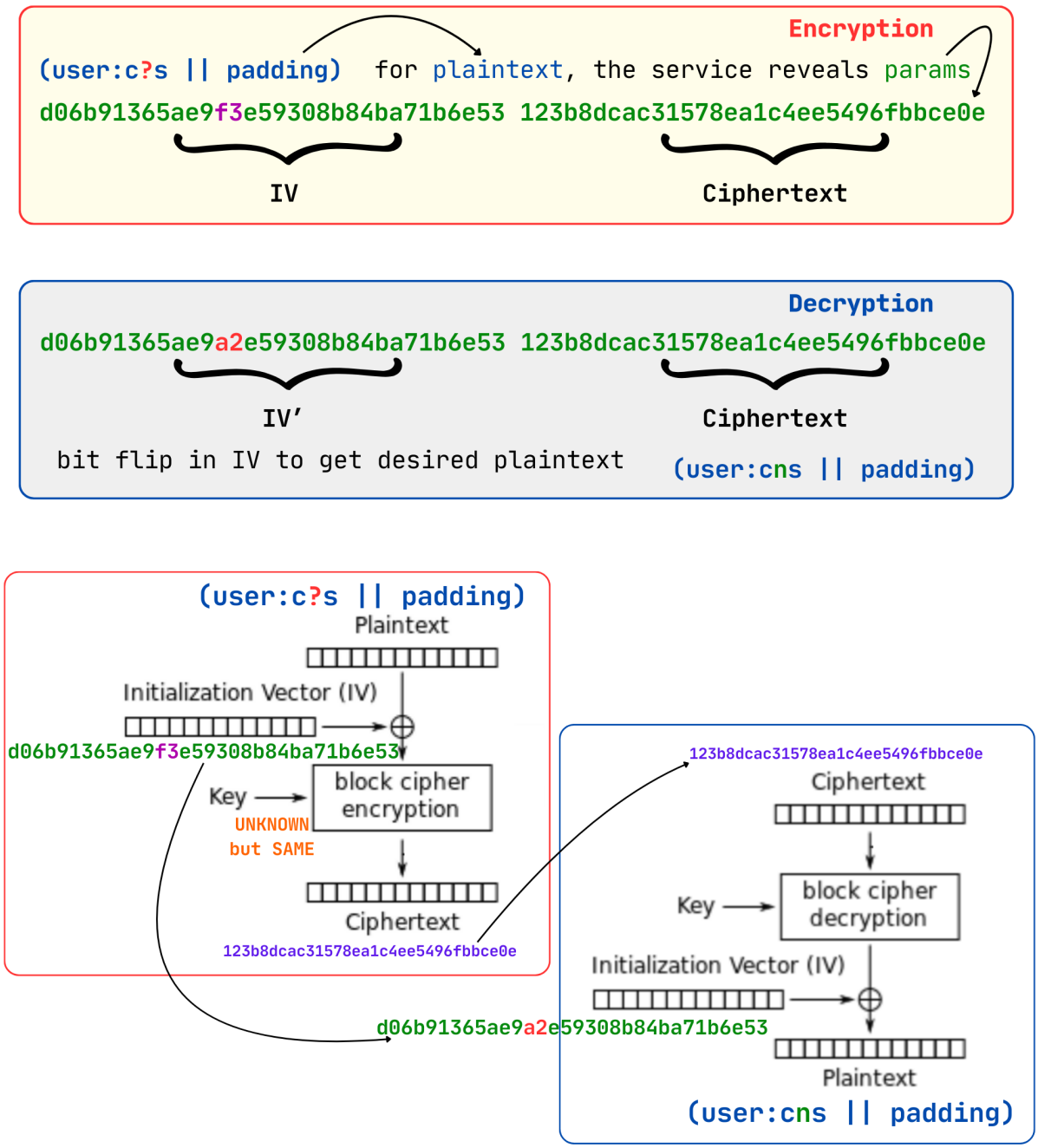

Figure 3: Logical Flow. Image Souce: easefilter.com/kb/understand-aes-encryption.htm